Call Me Maybe

Cours 6 — Implementer le constrained decoding

---

1. Initialisation — une seule fois au demarrage

```
import json
import numpy as np
from llm_sdk import Small_LLM_Model
model = Small_LLM_Model()   # charge Qwen3-0.6B
# Vocabulaire : str -> int
with open(model.get_path_to_tokenizer_file()) as f:
    tokenizer_data = json.load(f)
vocab       = tokenizer_data["model"]["vocab"]
id_to_token = {v: k for k, v in vocab.items()}  # int -> str
# Fonctions disponibles (depuis functions_definition.json)
functions = load_function_definitions('data/input/functions_definition.json')
# Pre-calculer les token IDs de chaque nom de fonction
func_tokens: dict[str, list[int]] = {}
for func in functions:
    func_tokens[func.name] = model.encode(func.name)[0].tolist()
# Resultat :
# func_tokens = {
#   "fn_add_numbers"   : [fn_id, _add_id, _numbers_id],
#   "fn_greet"         : [fn_id, _greet_id],
#   "fn_reverse_string": [fn_id, _reverse_id, _string_id],
#   "fn_get_square_root": [...],
#   "fn_substitute_string_with_regex": [...],
# }
```

2. La structure d'etat

```
from enum import Enum, auto
from dataclasses import dataclass, field
class JsonState(Enum):
    START       = auto()   # rien genere
    FUNC_NAME   = auto()   # nom de fonction token par token
    AFTER_NAME  = auto()   # nom termine, structure fixe jusqu'aux params
    PARAM_KEY   = auto()   # nom d'un parametre
    PARAM_VALUE = auto()   # valeur d'un parametre (number/bool)
    IN_STRING   = auto()   # dans une string, token par token
    PARAM_NEXT  = auto()   # apres une valeur : , ou }
    END         = auto()   # JSON termine
@dataclass
class GenContext:
    state         : JsonState
    func_tokens   : dict[str, list[int]]
    functions     : list                  # list[FunctionDef]
    chosen_func   : object | None = None  # FunctionDef une fois connue
    name_so_far   : list[int] = field(default_factory=list)
    params_done   : list[str] = field(default_factory=list)
    current_param : str | None = None
    current_value : str = ''              # valeur en cours de generation
    in_str_closed : bool = False
```

3. get_valid_tokens() — la fonction centrale

```
def get_valid_tokens(ctx: GenContext, vocab: dict[str, int]) -> list[int]:
    if ctx.state == JsonState.FUNC_NAME:
        n = len(ctx.name_so_far)
        valid = set()
        for func_name, token_ids in ctx.func_tokens.items():
            if len(token_ids) > n:
                if token_ids[:n] == ctx.name_so_far:  # cohérent ?
                    valid.add(token_ids[n])           # prochain token
        if not valid:
            raise ValueError('Aucun token valide pour FUNC_NAME')
        return list(valid)
    if ctx.state == JsonState.PARAM_KEY:
        all_params = list(ctx.chosen_func.parameters.keys())
        remaining  = [p for p in all_params if p not in ctx.params_done]
        next_param = remaining[0]
        param_ids  = model.encode(next_param)[0].tolist()
        n          = ctx.fixed_so_far
        return [param_ids[n]] if n < len(param_ids) else [vocab[':']]
    if ctx.state == JsonState.PARAM_VALUE:
        ptype = ctx.chosen_func.parameters[ctx.current_param].type
        if ptype == 'number' : return get_number_tokens(vocab, ctx)
        if ptype == 'string' : return [vocab['"']]  # debut de string
        if ptype == 'boolean': return [vocab['true'], vocab['false']]
    if ctx.state == JsonState.IN_STRING:
        quote_id = vocab.get('"')
        return [v for k, v in vocab.items() if v != quote_id]
    if ctx.state == JsonState.PARAM_NEXT:
        all_params = list(ctx.chosen_func.parameters.keys())
        remaining  = [p for p in all_params if p not in ctx.params_done]
        return [vocab[',']] if remaining else [vocab['}']]
    raise ValueError(f'Etat inconnu : {ctx.state}')
```

4. get_number_tokens() — generer un nombre

Un token peut contenir plusieurs caracteres. Par exemple le vocabulaire peut contenir un token '3}' ou '2,' — chiffre et separateur dans le meme token. Il faut les autoriser pendant PARAM_VALUE, pas seulement pendant PARAM_NEXT.

```
def get_number_tokens(vocab: dict[str, int], ctx: GenContext) -> list[int]:
    generated = ctx.current_value   # ex: '2' ou '26' ou '2.'
    has_dot   = '.' in generated
    has_exp   = 'e' in generated.lower()
    valid     = []
    for token, tid in vocab.items():
        t = token.strip()
        if not t: continue
        if not generated and (t[0].isdigit() or t[0] == '-'):
            valid.append(tid)           # debut : chiffre ou signe
        elif t[0].isdigit():
            valid.append(tid)           # continuer avec des chiffres
        elif t == '.' and not has_dot and not has_exp:
            valid.append(tid)           # un seul point decimal
        elif t in ('e','E') and not has_exp and generated[-1:].isdigit():
            valid.append(tid)           # notation scientifique
        elif t in (',', '}'):
            valid.append(tid)           # fin de valeur — CLE !
    return valid
```

5. update_state() — transition apres chaque token

```
def update_state(ctx: GenContext, next_id: int, next_token: str) -> None:
    if ctx.state == JsonState.FUNC_NAME:
        ctx.name_so_far.append(next_id)
        for func_name, token_ids in ctx.func_tokens.items():
            if ctx.name_so_far == token_ids:
                ctx.chosen_func = next(f for f in ctx.functions
                                       if f.name == func_name)
                ctx.state = JsonState.AFTER_NAME
                return
        # Pas encore complet -> reste en FUNC_NAME
    elif ctx.state == JsonState.PARAM_VALUE:
        ctx.current_value += next_token
        ptype = ctx.chosen_func.parameters[ctx.current_param].type
        if ptype == 'string':
            ctx.state = JsonState.IN_STRING  # le '"' ouvre la string
        elif next_token.strip() in (',', '}'):
            ctx.params_done.append(ctx.current_param)
            ctx.state = JsonState.PARAM_NEXT
    elif ctx.state == JsonState.IN_STRING:
        if next_token == '"':            # '"' fermant
            ctx.params_done.append(ctx.current_param)
            ctx.state = JsonState.PARAM_NEXT
    elif ctx.state == JsonState.PARAM_NEXT:
        if next_token.strip() == ',':
            ctx.state = JsonState.PARAM_KEY
            ctx.current_param = None
            ctx.current_value = ''
        elif next_token.strip() == '}':
            ctx.state = JsonState.END
```

6. Exemple complet — 'Greet shrek'

Tokens	Genere	Etat	Note
1-6	{ " name " : "	START -> FUNC_NAME	Structure fixe generee token par token
7	fn	FUNC_NAME	fn_greet et fn_add... encore possibles
8	_greet	FUNC_NAME	fn_greet confirme ! -> AFTER_NAME
9-N	" , " parameters " : { "	AFTER_NAME	Structure fixe jusqu'aux params
N+1	name	PARAM_KEY	Premier (et seul) param de fn_greet

Tokens	Genere	Etat	Note
N+2	" : "	PARAM_VALUE	Separateur, puis ouvre la string
N+3	shrek	IN_STRING	Valeur du parametre name
N+4	"	IN_STRING	Ferme la string -> PARAM_NEXT
N+5	}	PARAM_NEXT	Ferme parameters (dernier param)
N+6	}	END	Ferme l'objet principal

Resultat JSON : {"name": "fn_greet", "parameters": {"name": "shrek"}}

7. Cas complexe — fn_substitute_string_with_regex

Cette fonction a 3 parametres string. L'etat IN_STRING est traverse 3 fois — une fois par parametre. params_done permet de savoir quand mettre , ou } :

Parametre	Type	params_done avant	Token apres la valeur
source_string	string	params_done = []	, (il reste regex et replacement)
regex	string	params_done = ['source_string']	, (il reste replacement)
replacement	string	params_done = ['source_string','regex']	} (dernier parametre)

8. La boucle principale complete

```
def generate_function_call(prompt: str, ctx: GenContext,
                           vocab: dict[str, int],
                           id_to_token: dict[int, str]) -> FunctionCall:
    input_ids     = model.encode(prompt)[0].tolist()
    generated_json = ''
    eos_id        = model._tokenizer.eos_token_id
    while ctx.state != JsonState.END:
        # 1. Logits
        logits    = model.get_logits_from_input_ids(input_ids)
        logits_np = np.array(logits)
        # 2. Tokens valides
        valid_ids = get_valid_tokens(ctx, vocab)
        if not valid_ids:
            raise ValueError(f'Aucun token valide en etat {ctx.state}')
        # 3. Masque
        mask = np.full(len(logits_np), -np.inf)
        mask[valid_ids] = 0.0
        logits_np = logits_np + mask
        # 4. Selection
        next_id    = int(np.argmax(logits_np))
        next_token = id_to_token.get(next_id, '')
        # 5. Mise a jour
        input_ids.append(next_id)
        generated_json += next_token
        update_state(ctx, next_id, next_token)
        if next_id == eos_id:
            break
    result = json.loads(generated_json)
    return FunctionCall(prompt=prompt, **result)
```

9. Points critiques

encode()[0].tolist()	encode() retourne un tenseur 2D. Il faut [0].tolist() pour obtenir list[int].
get_logits : list[int] -> list[float]	Pas de tenseur. Convertir avec np.array() avant le masquage numpy.
, et } dans get_number_tokens	Un token peut contenir chiffre + separateur (ex: '3}'). Il faut les autoriser pendant PARAM_VALUE sinon on bloque une sortie valide.
IN_STRING 3 fois pour fn_substitute	3 parametres string = 3 passages dans IN_STRING. La boucle PARAM_KEY->PARAM_VALUE->IN_STRING->PARAM_NEXT tourne 3 fois.

Liste vide -> NaN	Si get_valid_tokens() retourne [], tous les logits sont a -inf. Apres softmax : NaN. Detecter ce cas et lever une ValueError.

Exercice

Q1. Prompt : 'What is the sum of 2 and 3?' -> fn_add_numbers(a, b: number). Apres que le modele a genere fn puis _add, quelles fonctions sont encore possibles ? Quel est le seul token valide suivant ?

Q2. Prompt : 'Replace all vowels in Programming is fun with asterisks' -> fn_substitute_string_with_regex(source_string, regex, replacement : tous string). Combien de fois l'etat IN_STRING est-il traverse ? Justifie.

Q3. Dans get_number_tokens(), pourquoi ajoute-t-on ',' et '}' comme tokens valides pendant PARAM_VALUE ? Ne sont-ils pas reserves a PARAM_NEXT ?

Reponses

Q 1	Apres fn + _add : la condition token_ids[:2] == [fn_id, _add_id] elimine fn_greet, fn_reverse_string, fn_get_square_root et fn_substitute_string_with_regex. Seule fn_add_numbers reste. Le seul token valide suivant est _numbers_id.
Q 2	IN_STRING est traverse 3 fois — une fois par parametre string. source_string : IN_STRING jusqu'au " fermant -> PARAM_NEXT -> , regex : idem -> PARAM_NEXT -> , replacement : idem -> PARAM_NEXT -> } (dernier param).
Q 3	Un token du vocabulaire peut contenir plusieurs caracteres. Par exemple '3}' ou '2,' sont des tokens valides : chiffre et separateur ensemble. Si on ne les autorise pas pendant PARAM_VALUE, on bloque ces tokens et le modele ne peut pas terminer proprement la valeur numerique.

A retenir

- encode()[0].tolist() -> list[int] | get_logits_from_input_ids(list[int]) -> list[float].
- get_valid_tokens() retourne les IDs valides pour l'etat courant.
- update_state() fait avancer la machine d'etats apres chaque token.
- IN_STRING est traverse autant de fois qu'il y a de parametres string.
- ',' et '}' sont autorises pendant PARAM_VALUE car un token peut les contenir avec un chiffre.

Prochaine etape

Cours 7 — Structure du projet : organiser ton code en modules, configurer uv, le Makefile, mypy, flake8, les type hints et les docstrings. Tout ce qu'il faut pour que le projet soit conforme aux exigences du sujet.